## Bayes, MAP, MLE

P(h|D) = P(D|h)P(h)/P(D) post = likelihood × prior / evidence

Total prob: P(D) = Σh P(D|h)P(h); P(S)=P(S|M)P(M)+P(S|¬M)(1−P(M))

MAP: argmax_h P(D|h)P(h) (drop P(D)). MLE: argmax_h P(D|h)

MLE = MAP if prior uniform. iid: P(D|h) = Π p(xi|h)

Classify: Ck = argmax p(x|Ck)P(Ck); posterior = score/Σ scores

Check: P(S|M)=0.5, P(M)=1/50000, P(S)=1/20: P(M|S)=0.0002

## Gaussian fit + Gaussian NB

μ = (1/n)Σxi $\sigma^2$ = (1/n)Σ(xi−μ)$^2$ (MLE: divide by n)

State "MLE, /n". Unbiased /(n−1) only if asked (= $\sigma^2$ × n/(n−1))

1-D: p(x) = 1/√(2π$\sigma^2$) × e^(−(x−μ)$^2$/(2$\sigma^2$))

Recipe per class k: prior P(Ck) = nk/n μk = mean of each feature over class-k points only table: x, x−μ, (x−μ)$^2$, sum, ÷ nk (do per feature) Σk = diag($\sigma1^2$, $\sigma2^2$), off-diagonals 0 (cond. independence)

Classify x: score_k = P(Ck) × Πj N(xj | μkj, $\sigma kj^2$); max wins

Posterior = score_k/Σ scores. Tiny numbers: compare ln scores

Full Gaussian: cov(x,y) = (1/n)Σ(x−μx)(y−μy); naive sets it 0

Params: naive K×2d + (K−1); full K(d + d(d+1)/2) + (K−1)

Check: x = −1,1,2,2,4: μ=1.6, Σ(x−μ)$^2$=13.2, $\sigma^2$=13.2/5=2.64

## Discrete NB + Laplace

P(xj=v|Ck) = count(v and Ck)/count(Ck)

C = argmax_k P(Ck) Πj P(xj|Ck)

Tennis (sunny,cool,high,strong): yes 0.0053 < no 0.0206: No

Laplace: P(w|C) = (count(w,C)+1)/(total words in C + |V|)

Why: one zero count makes the whole product 0

## Trees: entropy, IG, Gini, gain ratio

H(S) = −Σ pi log2 pi; 2 classes: max 1 at 50/50, pure = 0

H(S|A) = Σv (|Sv|/|S|) H(Sv); IG = H(S) − H(S|A); pick max

Gini = 1 − Σpi$^2$; GiniGain = Gini(S) − Σv (|Sv|/|S|)Gini(Sv)

SplitInfo = −Σv (|Sv|/|S|) log2(|Sv|/|S|); GR = IG/SplitInfo

Calc: log2(x) = ln(x)/ln(2) = ln x ÷ 0.6931; 0×log2(0) = 0

Check: H(9+,5−)=0.940, H(2,3)=0.971, Gini(9,5)=0.459

Tennis: IG Outlook 0.247 > Humidity 0.152; GR=0.247/1.577=0.156

Continuous: sort, threshold θ = midpoint where class changes

## Confusion matrix, ROC, AUC

Rows actual, cols predicted: [TP FN; FP TN]

Acc = (TP+TN)/all Prec = TP/(TP+FP) Rec = TPR = TP/(TP+FN)

FPR = FP/(FP+TN) Specificity = TN/(FP+TN) = 1 − FPR

F1 = 2PR/(P+R) Fβ = (1+β$^2$)PR/(β$^2$P + R)

ROC: x = FPR, y = TPR per threshold; add (0,0),(1,1); diagonal

AUC = area: 0.5 random, 1 perfect. Trapezoid Σ(x2−x1)(y1+y2)/2

Points below diagonal (TPR < FPR): worse than random

Fix: invert classifier (swap +/− decisions, or use 1 − score) (FPR,TPR) becomes (1−FPR, 1−TPR); new AUC = 1 − old AUC

Imbalanced: accuracy misleading, prefer precision-recall curve

Eg 20 pos, 80 neg, TP5 FN15 FP30 TN50: TPR=5/20=0.25, FPR=30/80=0.375

## Linear regression

h(x) = w0 + w1x1 + ... = w$^T$x (x0=1); MSE J = (1/n)Σ(h(xi)−yi)$^2$

∂J/∂w0 = (2/n)Σ(h(xi)−yi); ∂J/∂w1 = (2/n)Σ xi(h(xi)−yi)

GD: wj ← wj − α ∂J/∂wj, all j updated simultaneously

Normal eq: w = (X$^T$X)$^{-1}$X$^T$y

$R^2$ = 1 − SSE/SST; SSE = Σ(yi−ŷi)$^2$, SST = Σ(yi−ȳ)$^2$, MSE = SSE/n

Ridge J + α‖w‖$_2^2$ = αΣw$^2$; Lasso J + α‖w‖$_1$ = α Σ|w|

E[error] = Bias$^2$ + Variance + $\sigma^2$ (irreducible noise)

## Logistic regression, softmax

σ(z) = 1/(1+e^(−z)); σ(0)=0.5; σ(−z) = 1−σ(z); σ' = σ(1−σ)

z = w$^T$x + b = w1x1 + w2x2 + ... + b

σ(z) = p(y=+1|x); p(y=−1|x) = 1 − σ(z) = σ(−z); ±1: p(y|x) = σ(yz)

odds = p/(1−p) = e^z; log-odds ln(p/(1−p)) = z; z > 0: predict +1

Boundary w$^T$x + b = 0 (p = 0.5); 1-D: x* = −b/w

Log loss y∈{0,1}: −Σ[y ln ŷ + (1−y) ln(1−ŷ)]; ∂/∂z = ŷ − y

y∈{±1}: L = Σ ln(1 + e^(−y w$^T$x))

Softmax p(k|x) = e^(zk)/Σj e^(zj), zk = wk$^T$x + βk: K(d+1) params

Multiclass CE = −ln p(true class)

Why d+1 params: K=2, divide top and bottom by e^(z1): p(1|x) = 1/(1 + e^(z2−z1)) = 1/(1 + e^(−(w1−w2)$^T$x − (β1−β2))) = σ(w$^T$x + β), w = w1−w2, β = β1−β2: only differences, d+1 Add same (c, b) to both classes: probs unchanged, redundant General K: (K−1)(d+1) free params; K=2 gives d+1

Check: σ(2) = 0.881; −ln(0.8) = 0.223

## Bootstrap, bagging

n draws with replacement. P(not picked in 1 draw) = 1 − 1/n

P(never picked in n draws) = (1 − 1/n)^n, tends to 1/e = 0.368

P(selected) = 1 − (1 − 1/n)^n, tends to 1 − 1/e = 0.632

n=10: 0.651, n=100: 0.634, n=1000: 0.632

OOB 36.8% = free validation set (oob_score)

Average of B iid models: Var = $\sigma^2$/B

## AdaBoost (labels ±1)

err_t = Σ w_i [h_t(x_i) ≠ y_i] / Σ w_i

α_t = ½ ln((1 − err_t)/err_t); err 1/3: α = ½ ln 2 = 0.347

w_i ← w_i e^(−α_t y_i h_t(x_i)): wrong × e^α, right × e^(−α)

Then renormalise so Σ w_i = 1

Equal start, k wrong of n: wrong = 1/(2k), right = 1/(2(n−k))

Final H(x) = sign(Σt α_t h_t(x))

Notebook/sklearn variant α = ln((1−ε)/ε): say which you use

## Perceptron, neuron

ŷ = 1 if Σ wjxj > 0 else −1 (x0 = 1 is bias); labels ±1

Step perceptron: w ← w + η(y − ŷ)x; wrong gives y − ŷ = ±2

Slide SGD eg (ŷ = w.x, J = (y−ŷ)$^2$): w ← w + 2η(y − ŷ)x

η = 0.5: w ← w + (y−ŷ)x; w (0,0,0) to (1,1,1) to (−2,4,−5)

Exam: write the rule you use before applying it

Sigmoid + BCE step: w ← w − η(ŷ − y)x

MLP params m(d+1) + k(m+1); dense layer n_in×n_out + n_out

## CNN sizes and params

O = (N + 2P − F)/S + 1; P=0, S=1: O = N − F + 1

Pool 2×2 stride 2: O = N/2, depth unchanged, 0 params

Conv params = K × (F×F×C_in + 1); C_in = depth of input

Output O×O×K. Check: 32×32×3, 6 of 5×5: 28×28×6, 456 params

Swap question: write both configs as formulas, subtract, sign

Swap pool1: (p−s+1)/2, s1 to smaller s2: INCREASE pool2: (p−s1−2s2+3)/4; swapped − correct = (s2−s1)/4: DECREASE conv1 f1(3s$^2$+1): DECREASE; conv2 f2(f1s$^2$+1): INCREASE total diff = f1(s1$^2$−s2$^2$)(f2−3) > 0: INCREASE (big kernel, f1 ch)

## Attention

Attention(Q,K,V) = softmax(QK$^T$/√dk) V, softmax per row

score q.k/√dk, weights α sum to 1, output Σ αi vi

RNN params dn + n$^2$ + n; LSTM 4(n(n+d) + n) (input d, hidden n)

2 keys: α1 = 1/(1 + e^(s2−s1)). Causal mask: n(n+1)/2 of n$^2$

## k-means + distances

Init k centres. Repeat: 1) assign each point to nearest centre

2) update: centre = mean of its points (x̄, ȳ)

Converged when assignments unchanged (centres stop moving)

Table: point | d to c1 | d to c2 | cluster; 3 dp for near-ties

SSE = Σ‖xi − μ(ci)‖$^2$ never increases; only a local optimum

Euclid d = √((x1−x2)$^2$ + (y1−y2)$^2$); (2,8) to (5,5) = √18 = 4.243

Cosine = (a.b)/(|a||b|): (1,2),(2,1) gives 4/5 = 0.8

Silhouette s = (b−a)/max(a,b); k-means++: P(x) ∝ D(x)$^2$

## Calculator keys (fx-97SG X)

Setup: SHIFT MENU (SETUP), Input/Output, 2 MathI/DecimalO

Stats: MENU, Statistics icon, =, 1 (1-Variable); x =, x =, ...

OPTN 3 (1-Variable Calc): x̄, $\sigma^2$x (÷n, USE), σx, $s^2$x (÷n−1)

Clear data: OPTN 2 2 (Delete All); exit stats: MENU, Calculate

log2(x) = log(x) ÷ log(2) (no log_a key); e^x = SHIFT ln

Store: STO (−) puts Ans in A; use: ALPHA (−). Decimal: SHIFT =

Bracket: e^(−(x−μ)$^2$/(2$\sigma^2$)), 1/(1+e^(−z)). Reset: SHIFT 9 3 =

## Method answers

Unsure of a formula or keys? Run these inputs first. Same result = method right; now use the exam numbers.

GNB μ0=(1.6,4.4), μ1=(3.4,2.6), $\sigma^2$=2.64, x=(2,4): coef 1/√(2π×2.64) = 0.2455 c0: e^(−0.16/5.28)=0.9702, dens 0.2382, ×2 feats: 0.0567 c1: e^(−1.96/5.28)=0.6899, dens 0.1694, ×2 feats: 0.0287 P(c0|x) = 0.0567/(0.0567+0.0287) = 0.6641: class 0

Bayes P(D)=.01,P(+|D)=.9,P(+|¬D)=.05: .009/.0585 = 0.1538

IG S[29+,35−] H=0.9937; A1 [21,5][8,30]: H|A1=0.7278 IG(A1)=0.2659 > IG(A2 [18,33][11,2])=0.1214: split A1

Logit w=(0.8,−0.5), b=−1, x=(3,2): z=0.4, p=0.5987

AdaBoost 10 ex w=0.1, 3 wrong: ε=0.3, α=0.4236 e^α=1.5275, e^−α=0.6547, Z=0.9165: wrong 1/6, right 1/14

Backprop slide: x=(1,1,2), y=2, z=(1,.5,.5), ŷ=1, J=1 ∂J/∂W2 = 2(ŷ−y)z = (−2,−1,−1); hidden δ = −2×1×0.25 ∂J/∂W1 rows (−.5,−.5,−1); η=0.1: ŷ=1.4638, J=0.2875

Attn q=(1,0,2,1): q.k = 1,3,4; ÷√4 = 0.5,1.5,2 softmax (0.1220,0.3315,0.5465); v=(10,2,6): out 5.1618

## How to answer

Explain: claim, reason (because), formula or example, so-what

Compare: what each does, pro and con each, when to use

Justify: choice, deciding property, what fails otherwise

Name key terms. ~1.7 min/mark; method marks count

## Explainers

**Why do we need a validation set?** To choose models and hyperparameters (α, depth, epochs) and detect overfitting on unseen data. Training error is optimistic, so cannot pick models.

**Split validation from the train or the test set?** From the TRAINING set. Test is used once at the end; tuning on it leaks test info (data leakage), so the reported score is too optimistic.

**Validation vs cross-validation?** Hold-out: one split, cheap, high variance, val data never trained on. k-fold CV: train on k-1 folds, validate on 1, rotate, average; all data validated once, lower variance, k× cost.

**When stratified or time-series CV?** Stratified keeps class ratios per fold (imbalanced data). Time data: validate on a later period, no shuffling.

**Why are d+1 params enough for binary logistic?** Softmax K=2, divide by e^(w1·x+β1): p(1|x) = 1/(1+e^(-(w1-w2)·x-(β1-β2))). Only differences matter: w = w1-w2, β = β1-β2, so d+1; p(2|x) = 1-p(1|x).

**Why is K(d+1) redundant?** Adding the same (c, b) to all classes leaves softmax unchanged, so fix one class at 0: (K-1)(d+1) free params.

**Why does a bootstrap sample hold ~63.2% of points?** Missed in one draw w.p. 1-1/n, in n draws (1-1/n)^n → 1/e ≈ 0.368, so P(chosen) = 1-(1-1/n)^n ≈ 0.632.

**What is the out-of-bag (OOB) set for?** The ~36.8% not drawn are unseen by that model, so they give a free validation (OOB) error estimate.

**ROC points below the diagonal: problem and fix?** TPR < FPR, worse than random (AUC < 0.5): scores are inverted. Invert the classifier (use 1-score): (FPR, TPR) becomes (1-FPR, 1-TPR), AUC becomes 1-AUC.

**Has k-means converged?** Only if an assignment step moves no point, so centres stop moving. If any point switched or a centre moved: No, iterate again.

**Why do total conv params rise when the big kernel moves to layer 2?** Params = f(C_in $k^2$ + 1). Layer 1 has C_in = 3, layer 2 has C_in = f1 > 10, so big kernels cost more there: change = f1($s1^2$-$s2^2$)(f2-3) > 0.

## Fitting, bias-variance, regularisation

**Overfitting vs underfitting?** Overfit: train error low, val high, fits noise (high variance). Underfit: both high, too simple (high bias); fix: more features or complexity, less regularisation.

**How to reduce overfitting?** More data or augmentation, simpler model, L1/L2 regularisation, pruning, dropout, early stopping, bagging.

**Bias-variance trade-off?** Error = bias$^2$ + variance + noise. More complexity lowers bias, raises variance; choose the complexity at minimum validation error.

**How does regularisation help?** Loss + α‖w‖ penalises big weights, so the fit is smoother: variance falls, bias rises a little. Larger α = simpler model.

**L1 (Lasso) vs L2 (Ridge)?** L2 αΣ$w^2$ shrinks all weights smoothly, rarely to exactly 0. L1 αΣ|w| sets some w exactly 0 = feature selection, sparse model.

**Learning rate too big or too small?** Too small: tiny steps, slow. Too big: overshoots the minimum, loss oscillates or diverges.

**Why gradient descent, not the closed form?** Normal equation $(X^TX)^{-1}X^Ty$ costs ~O($d^3$) and needs all data in memory; GD scales to big d, n and works where no closed form exists (logistic, NN).

**Parameter vs hyperparameter?** Parameters are learned in training (w, b). Hyperparameters are set before, chosen on validation (α, k, depth, η).

## Metrics

**Why is accuracy poor on imbalanced data?** Predicting all-majority (99% negative) scores 99% yet finds no positives. Use precision, recall, F1, PR-AUC; class weights, resampling.

**Precision vs recall, when each?** Precision TP/(TP+FP) when false alarms are costly (spam filter). Recall TP/(TP+FN) when misses are costly (cancer screening, fraud).

**Effect of raising the threshold?** Fewer predicted positives: FP fall so precision usually rises, FN rise so recall falls.

**Why is F1 a harmonic mean?** F1 = 2PR/(P+R) is pulled to the smaller value, so P=1, R=0.01 gives F1 ≈ 0.02 (arithmetic mean 0.505). Both must be good.

**What do ROC and AUC show?** ROC = TPR vs FPR over all thresholds; diagonal random, top-left perfect. AUC = P(random positive scored above random negative); 0.5 random, 1 perfect.

**ROC vs PR curve?** FPR divides by the many negatives, so ROC looks optimistic when positives are rare. PR curve ignores TN, so it is more informative on imbalanced data.

## Bayes and naive Bayes

**Generative vs discriminative?** Generative models p(x|C)P(C), can generate x (NB, GAN); ok with little data, strong assumptions. Discriminative models p(C|x) directly (logistic, trees, NN); better with much data.

**Naive Bayes assumption and why?** Features conditionally independent given the class: p(x|C) = Π p(xj|C). d easy 1-D estimates instead of one d-dim joint; Gaussian NB covariance is diagonal.

**Why does NB work though independence is false?** Only needs the right class to score highest (argmax), not exact probabilities. Fast, little data.

**Why Laplace smoothing?** An unseen word gives P(w|C) = 0 and the product zeroes the whole class. (count+1)/(total+|V|) keeps every probability > 0 and still sums to 1.

**MLE vs MAP?** MLE = argmax P(D|h), data only. MAP = argmax P(D|h)P(h), adds a prior. Uniform prior: MAP = MLE. MAP safer with little data.

## Decision trees

**What does entropy measure?** Impurity of class labels, H = -Σ p log2 p: 0 for a pure node, max (1 bit for 2 classes) at 50/50. Gini 1-Σ$p^2$ behaves alike.

**What is IG and how does ID3 use it?** IG = H(parent) - Σ (|Sv|/|S|) H(Sv), the entropy drop from a split. ID3 greedily splits on the max-IG attribute, recurses.

**Why does IG favour many-valued attributes?** An ID or date splits into tiny pure subsets, max IG but useless on new data. Gain ratio = IG / SplitInfo penalises many branches.

**Why do trees overfit and how to stop it?** They can split until every training row is right, fitting noise (high variance). Pre-prune (max depth, min leaf size), post-prune on validation, or use RF.

## Ensembles

**Why do ensembles work?** Combining diverse models cancels independent errors. Base models must beat chance (each 45% accurate: more voters is worse).

**Bagging vs boosting?** Bagging: parallel models on bootstrap samples, equal vote, cuts VARIANCE (RF). Boosting: sequential, each focuses on past errors, α-weighted vote, cuts BIAS; sensitive to noisy labels.

**Why does a random forest decorrelate trees?** Each split tries a random feature subset (≈√d), so one strong feature cannot dominate every tree; averaging less correlated trees cuts more variance.

**How does AdaBoost reweight?** ε = sum of weights of wrong examples; α = ½ ln((1-ε)/ε); wrong × e^α, right × e^(-α), renormalise. Next learner focuses on hard cases; predict sign(Σ αt ht(x)).

**Stacking vs blending?** Stacking trains a meta-model on out-of-fold predictions of the base models; blending trains it on a held-out set (simpler, uses less data).

## Neural networks

**Why a non-linear activation?** Else layers collapse to one linear map (W2W1x): no XOR, no curves.

**Vanishing gradient and fixes?** Backprop multiplies many derivatives (σ' ≤ 0.25), so early layers get ~0 gradient and stop learning. Fix: ReLU, He init, batch norm, skip connections, LSTM.

**How does dropout reduce overfitting?** Drops random units in training so none rely on others; like averaging many sub-nets. Test: all units, × keep prob α.

**Batch vs SGD vs mini-batch?** Batch: all data per update, stable, slow. SGD: 1 sample, fast, noisy zig-zag. Mini-batch: balance, the default.

## CNN

**Why CNN over FNN for images?** FNN: a weight per pixel per neuron, ignores layout. CNN: local connectivity + weight sharing = far fewer params, translation invariance, feature hierarchy.

**What is weight sharing?** One filter slides over all positions: params = k×k×C_in per filter, not image size; finds a pattern anywhere.

**Purpose of pooling?** Downsample (2×2 halves H, W): less compute, robust to small shifts; no weights.

**Effect of padding and stride?** Out = (n + 2p - k)/s + 1. Padding keeps size and border info; bigger stride shrinks output and compute.

**Why ResNet skip connections?** Block outputs F(x) + x: learns only the residual; gradient flows straight through the + to early layers, so 100+ layers train.

## RNN, attention, transformer

**RNN vs LSTM?** RNN gradient over T steps ~ W^T vanishes or explodes: forgets long context. LSTM forget/input/output gates + additive cell state keep gradients.

**Why self-attention over RNN?** Every token links directly to every other (path length 1), all positions in parallel, no vanishing through time. Cost O($n^2$).

**Why scale by √dk?** q·k sums dk terms, spread grows with dk; √dk stops softmax saturating (tiny gradients).

**Why multi-head?** Each head has its own Q, K, V projections, so each learns a different relation (syntax, coreference); concat × W^O.

**Why positional encoding?** Attention is order-blind; added sin/cos position vectors give word order.

**Why mask the decoder?** Token t must not see the future it predicts: scores j > t set to -∞.

**BERT vs GPT?** BERT: encoder-only, bidirectional, masked words: understanding. GPT: decoder-only, causal, next token: generation.

## Generative AI

**GAN, VAE, diffusion?** GAN: generator fakes from noise vs discriminator (real/fake), min-max. VAE: encoder gives μ, σ, z = μ + σε, decoder rebuilds. Diffusion: learn to remove noise step by step.

**Why RLHF after fine-tuning?** LLM = pre-train (self-sup.), SFT (sup.), RLHF. SFT copies demos, cannot rank answers; human rankings train a reward model, PPO maximises reward minus KL.

## Unsupervised

**k-means weaknesses?** Must pick k; init-sensitive local optimum (restarts, k-means++); assumes round equal-size clusters; outliers; needs scaling.

**How to choose k?** Elbow: plot SSE vs k, pick the bend (SSE always falls). Silhouette s = (b-a)/max(a,b): pick k with highest average. Or domain knowledge.

**k-means vs hierarchical?** k-means: k in advance, fast, random init. Agglomerative: merge closest pair, dendrogram cut at any k, deterministic, O($n^2$).

**What does PCA do, why centre?** Projects onto top-k orthogonal max-variance directions (covariance eigenvectors by λ). Centre so Σ = (1/n)$X^TX$; var explained = Σtop λ / Σλ.

**Why embeddings, why cosine?** One-hot is huge, sparse, all words equally far. Dense learned vectors put similar meanings close; cosine a·b/(‖a‖‖b‖) compares direction, ignores length.